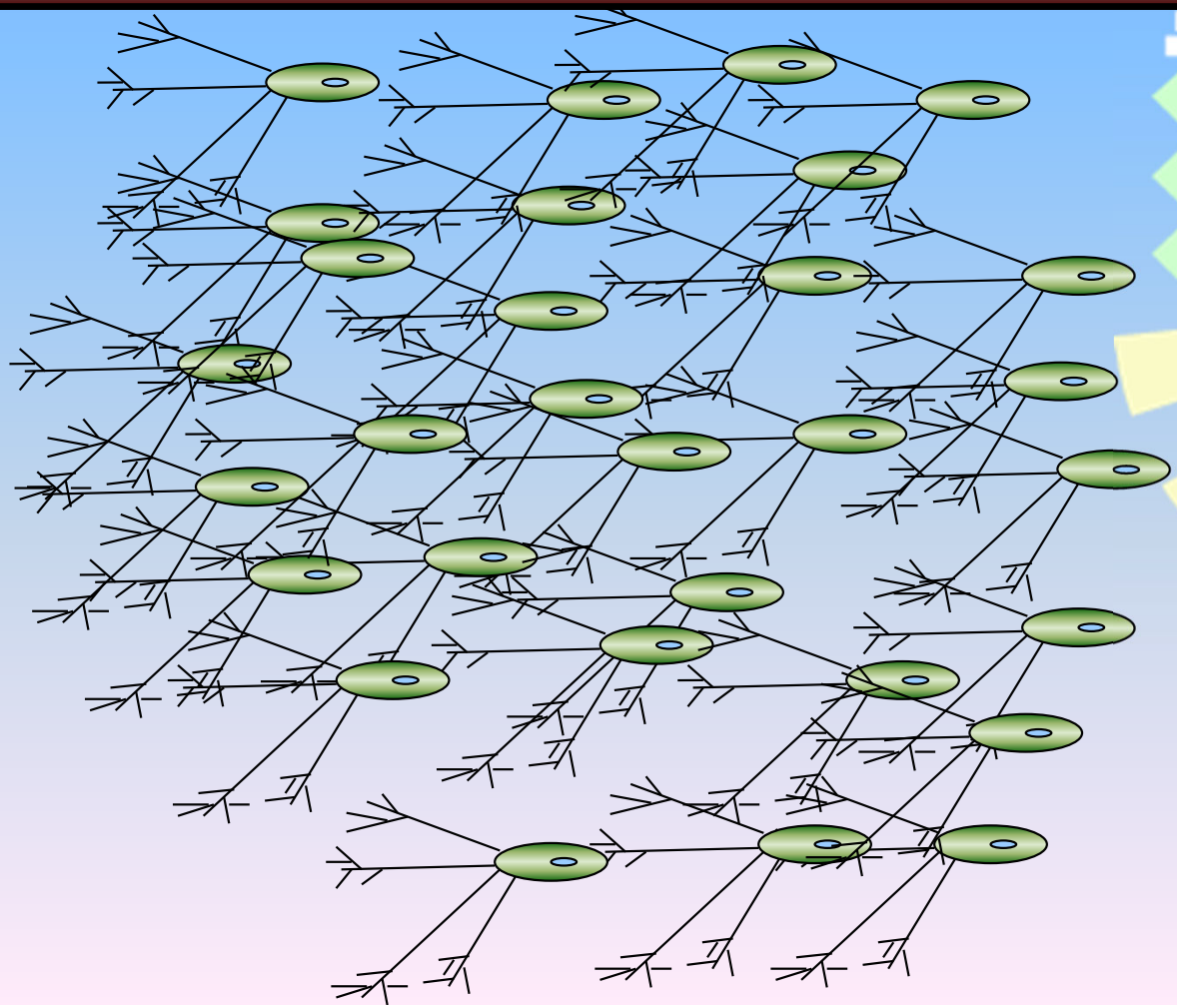
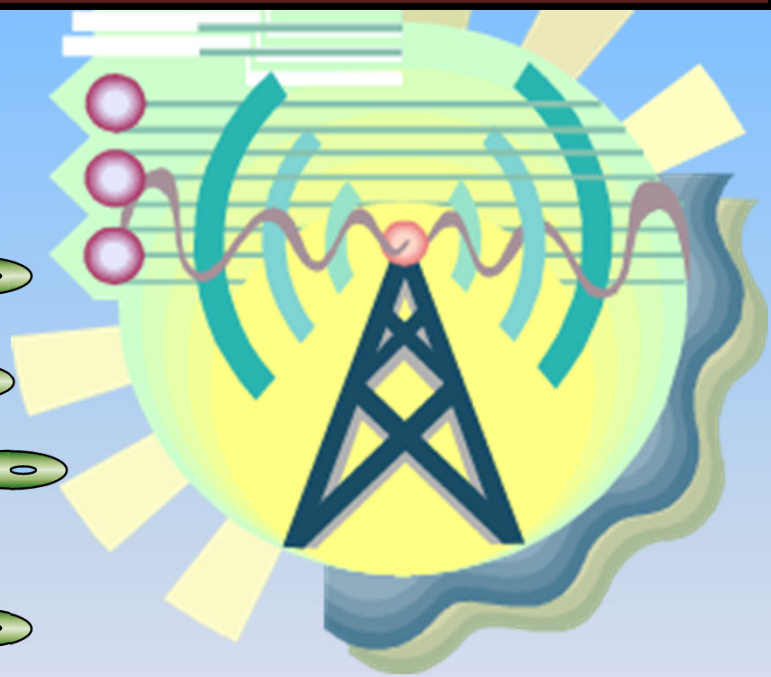
Neural Networks

Part 2





n.h.reyes@massey.ac.nz

Neural Networks

- [x] Introduction to Multi-layer Feed-forward Network
- [x] Evaluating a Feed-forward Network
- [x] Training a Feed-forward Network
- [x] Theory (Derivation of Formulas)
- [ ] Review of Formulas, Variations in Weight Updates
- [ ] Training Tips
- [ ] Encoding Inputs, Softmax units
- [ ] Performance Measure, Over-fitting problem

Applications, Tutorial Questions

Review of Formulas, Variations in Weight Updates

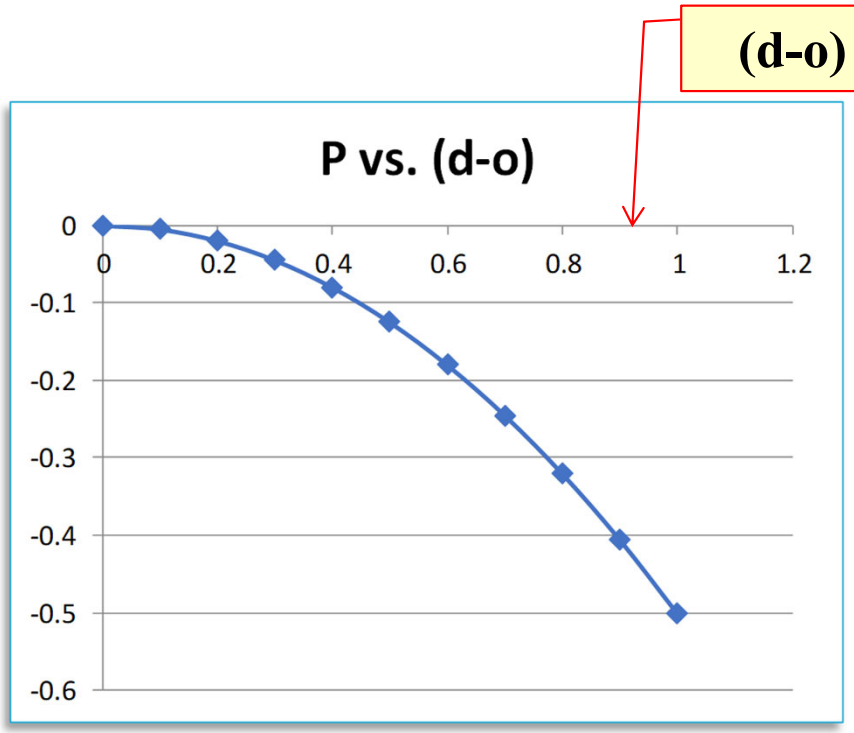


Gradient Ascent

Performance Function

$$P = (-1/2) * ((desired_output - actual_output)^2)$$

desired output	actual output	difference	Performance
1	1	0	0
1	0.9	0.1	-0.005
1	0.8	0.2	-0.02
1	0.7	0.3	-0.045
1	0.6	0.4	-0.08
1	0.5	0.5	-0.125
1	0.4	0.6	-0.18
1	0.3	0.7	-0.245
1	0.2	0.8	-0.32
1	0.1	0.9	-0.405
1	0	1	-0.5



When training the network, we would like to maximise P

Gradient Ascent

Performance Function

P = (-1/2) * ((desired_output – actual_output)^2)

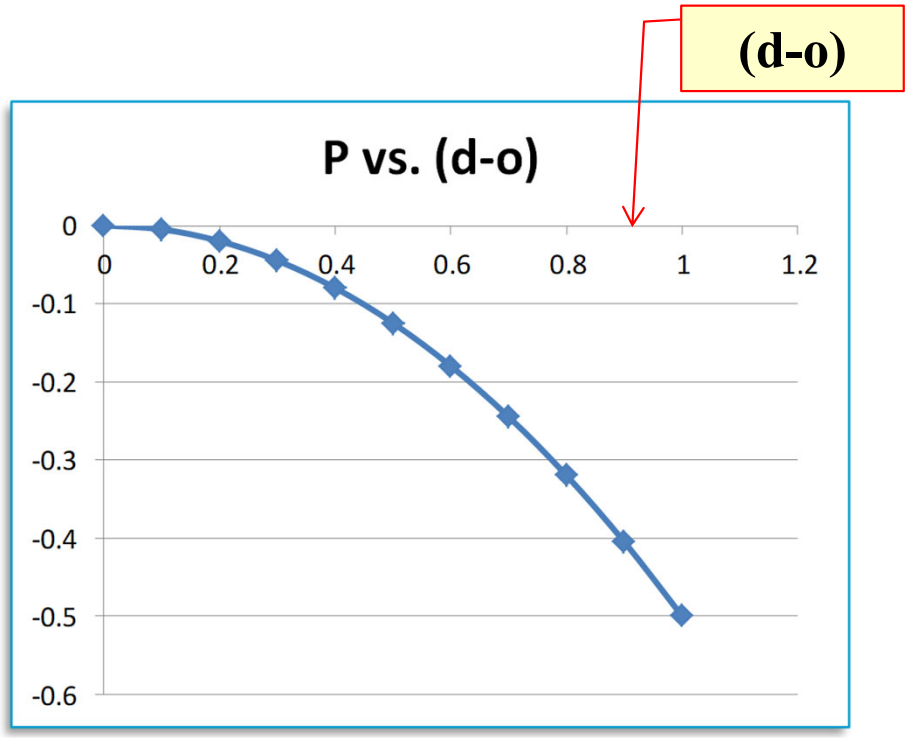
desired output	actual output	difference	Performance
1	1	0	0
1	0.9	0.1	-0.005
1	0.8	0.2	-0.02
1	0.7	0.3	-0.045
1	0.6	0.4	-0.08
1	0.5	0.5	-0.125
1	0.4	0.6	-0.18
1	0.3	0.7	-0.245
1	0.2	0.8	-0.32
1	0.1	0.9	-0.405
1	0	1	-0.5

desired output-actual output

$$\frac{dP}{do} = (d - o)$$

$$\Delta w \ \alpha \ \frac{\partial \mathrm{P}}{\partial w}$$

$$w = w + \eta \frac{\partial \mathrm{P}}{\partial w}$$



When training the network, we would like to maximise P

Error delta at the output node



Gradient Ascent

Error delta at Output node k:

$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

$O_k - activation\ function\ of\ output\ node\ k$
$p_r - weighted\ input\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

$\frac{dP}{do}$ - derivative of the performance function with respect to the actual output of the network

$\frac{dO_k}{dp_r}$ - derivative of the activation function at output node k, with respect to its weighted inputs



Gradient Ascent

Error delta at Output node k:

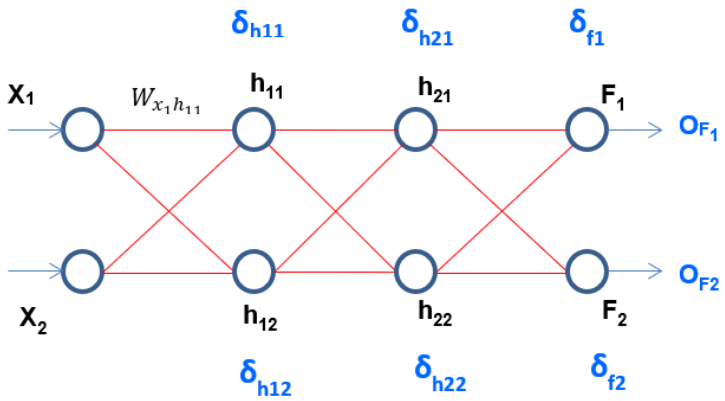
$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

$O_k - activation\ function\ of\ output\ node\ k$
$p_r - weighted\ input\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

Example:

We can use the error delta formula to compute for error deltas at output nodes F_1 and F_2.

Compute for: δ_{F_1} and δ_{F_2}



Updating weights leading into the output node(s).



Gradient Ascent

$$\delta_k = \underbrace{(t_k - O_k)}_{\frac{dP}{do}} \underbrace{Ok(1 - O_k)}_{\frac{dO_k}{dp_r}}$$

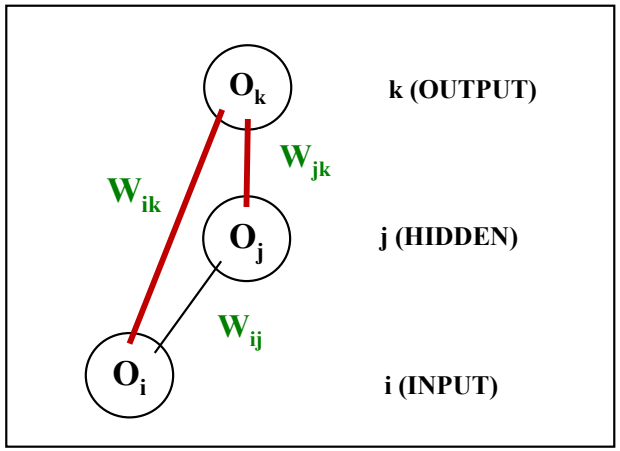
To adjust weights leading into an output unit:

$$W_{jk} = W_{jk} + \eta \delta_k O_j$$

$\eta - learning\ rate$
k – *output unit*
J – *node that is directly connected at output unit k*
$\delta_k - error\ signal\ at\ output\ unit\ k$
$O_j - output\ of\ unit\ j$



Gradient Ascent



Example #1:

To adjust weights leading into the output unit k, (i.e. W_{jk} and W_{ik}), we can use the formulas below:

$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

$$W_{jk} = W_{jk} + \eta \delta_k O_j$$

$$W_{ik} = W_{ik} + \eta \delta_k O_i$$

$\eta - learning_rate$
$\delta_k - error_signal$
$O_j - output_of_unit_j$
$O_i - output_of_unit_\boldsymbol{i}$

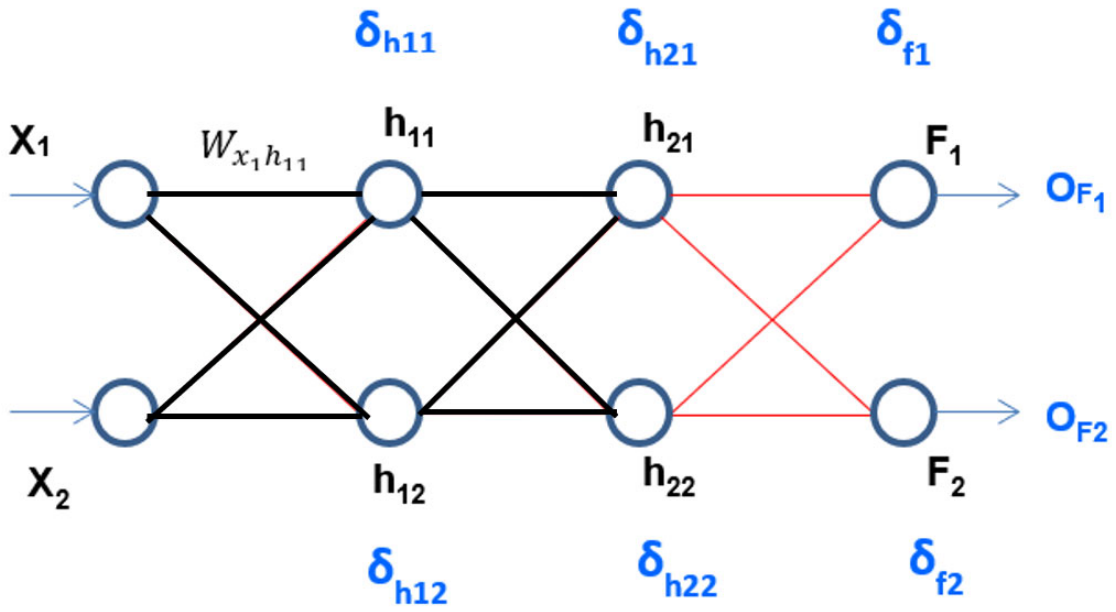
Gradient Ascent

Example #2:

In the figure below, all weights attached to the red-coloured links can be adjusted using the general weight-update formula represented by W_{jk}.

$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

$$W_{jk} = W_{jk} + \eta \delta_k O_j$$



Error delta at the hidden node(s)



Gradient Ascent

Error delta at a hidden node:

$$\frac{dO_j}{dp_r}$$

error signals back-propagated to node j

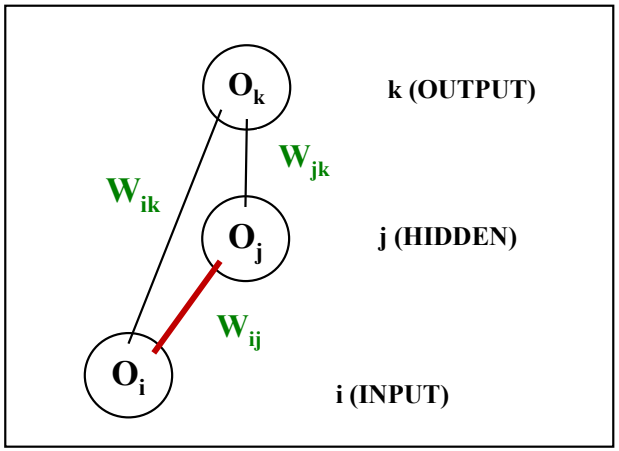
Weights of the links carrying the *back-propagated error signals*

$$\delta_j = Oj(1 - O_j)\sum_k \delta_k W_{jk}$$

$\eta - learning\ rate$
k – *output unit*
J – *hidden node that is directly connected at node k (which is residing at a higher layer)*
$\delta_k - error\ delta\ at\ node\ k$
$O_j - output\ of\ unit\ j$



Gradient Ascent



Example#1:

We can compute the error delta at hidden node j in the figure on the right.

$$\frac{dO_j}{dp_r}$$

error signals back-propagated to node j

Weights of the links carrying the *back-propagated error signals*

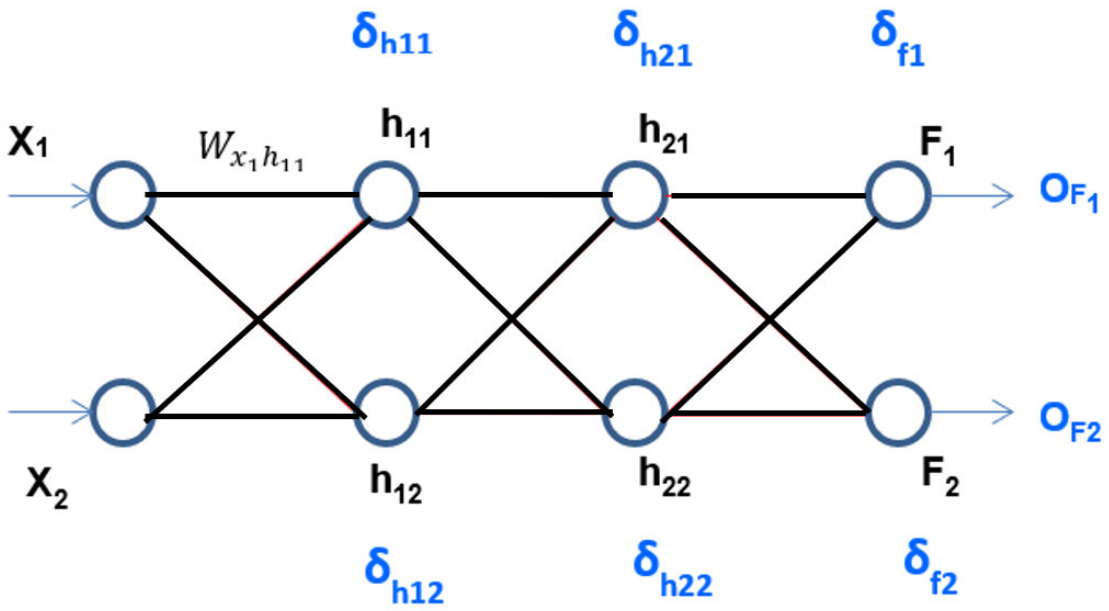
$$\delta_j = Oj(1 - O_j)\sum_k \delta_k W_{jk}$$

$\eta -$ *learning rate*
k – output unit
J – hidden node that is directly connected at node k (which is residing at a higher layer)
$\delta_k -$ *error delta at node k*
$O_j -$ *output of unit j*

Gradient Ascent

Example#2:

In the figure below, we can compute the error deltas at hidden nodes h_{21}, h_{22}, h_{11} and h_{12}, using the general error delta formula below.



$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

Updating weights leading into hidden node(s)



Gradient Ascent

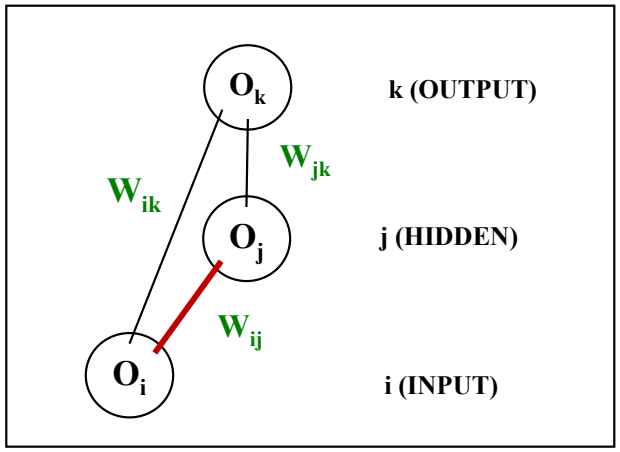
To adjust weights leading into a hidden unit, W_{ij}:

$$\delta_j = Oj(1 - O_j)\sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta\delta_j O_i$$

$\eta - learning_{rate}$
J – *hidden node*
$\delta_j - error\ signal\ at\ node\ j$
i – *node that is directly connected to hidden node j; node j is at a higher layer than node i*
$O_i - output\ of\ node\ i$

Gradient Ascent



Example #1:

Refer to the figure. To adjust the weight W_{ij}, leading into the hidden unit j, we can apply the formula below.

$$\delta_j = Oj(1 - O_j)\sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta\delta_j O_i$$

$\eta - learning_{rate}$
J – *hidden node*
$\delta_j - error_signal\ at\ node\ j$
i – *node that is directly connected to hidden node j;*
$O_i - output_of_unit_i$

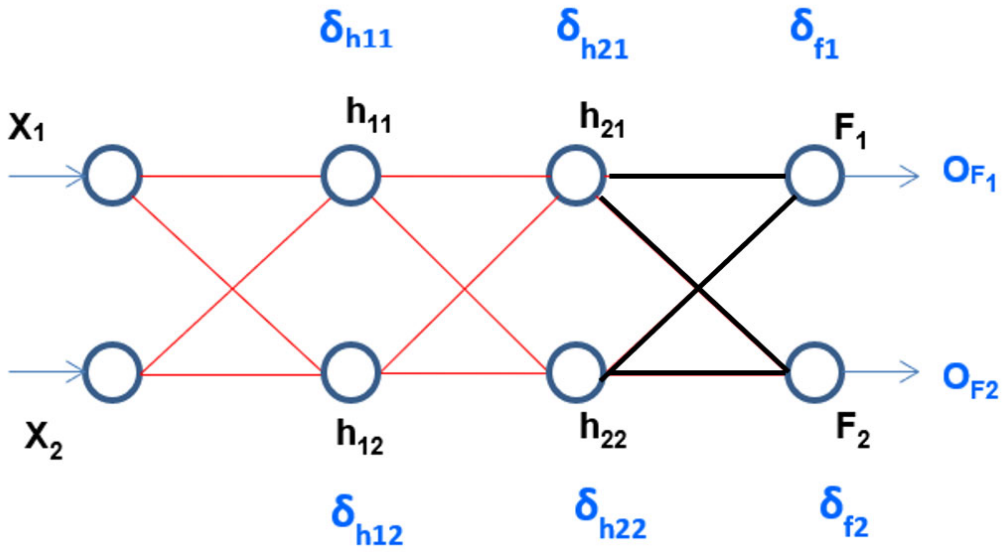
Gradient Ascent

Example #2:

Refer to the figure. To adjust all the weights leading into the hidden nodes (*i.e.* the weights attached to the red-coloured links), we can apply the general weight-update formula below.

$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta \delta_j O_i$$



$\eta - learning_{rate}$
J – *hidden node*
$\delta_j - error_signal\ at\ node\ j$
i – *node that is directly connected to hidden node j;*
$O_i - output_of_unit_i$

Gradient Ascent

Summary of Weight Update Formulas

$$\frac{dP}{do} = (d - o)$$

$$\Delta w \ \alpha \ \frac{\partial P}{\partial w}$$

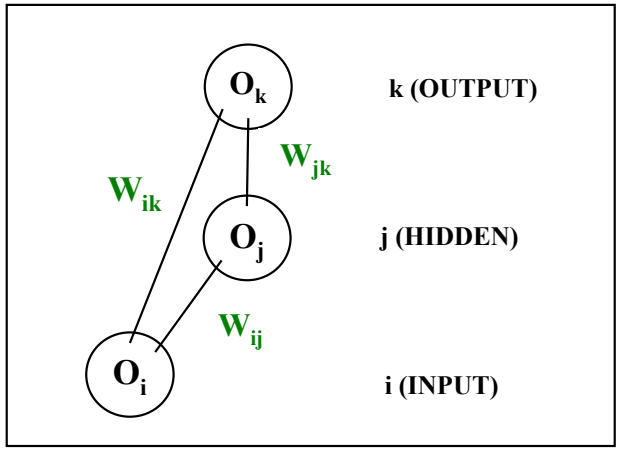
$$w = w + \eta \frac{\partial P}{\partial w}$$

To adjust weights leading into an output unit, W_{jk}:

$$\delta_k = (t_k - O_k)Ok(1 - O_k)$$

$$W_{jk} = W_{jk} + \eta \delta_k O_j$$

$$W_{ik} = W_{ik} + \eta \delta_k O_i$$



$\eta - learning_rate$
$\delta_k - error_signal$
$O_j - output_of_unit_j$

To adjust weights leading into a hidden unit, W_{ij}:

$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta \delta_j O_i$$

$\eta - learning_rate$
$\delta_j - error_signal$ at unit j
$O_i - output_of_unit_i$

Variations of Backpropagation

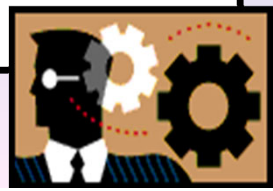


Backpropagation Training

On-line / Continuous Update/Stochastic Gradient Descent : Method 1

Iterative minimization of error over training set

1. Put one of the training patterns to be learned on the input units.
2. Find the values for the hidden unit and output unit.
3. Find out how large the error is on the output unit.
4. Use one of the back-propagation formulas to adjust the weights leading into the output unit.
5. Use another formula to find out error for the hidden layer unit.
6. Adjust the weights leading into the hidden layer unit via another formula.
7. Repeat steps 1 thru 6 for the second, third patterns,…



Backpropagation Training

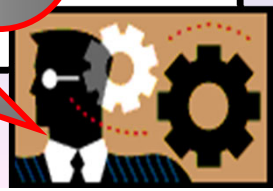
On-line / Continuous Update/Stochastic Gradient Descent : Method 1

Iterative minimization of error over training set

1. Put one of the training patterns to be learned on the input units.
2. Find the values for the hidden unit and output unit.
3. Find out how large the error is on the output unit.
4. Use one of the back-propagation formulas to adjust the weights leading into the output unit.
5. Use another formula to find out error for the hidden layer unit.
6. Adjust the weights leading int another formula.
7. Repeat steps 1 thru 6 for the

The weight changes are applied after each pattern is presented to the network.

This is the method that we used in solving the tutorial problems!



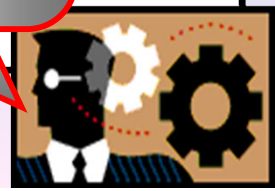
Backpropagation Training

On-line / Continuous Update/Stochastic Gradient Descent : Method 2

Iterative minimization of error over training set

1. Put one of the training patterns to be learned on the input units.
2. Find the values for the hidden unit and output unit.
3. Find out how large the error is on the output unit.
4. Use another formula to find out error for the hidden layer unit.
5. Use one of the back-propagation formulas to adjust the weights leading into the output unit.
6. Adjust the weights leading into the hid… another formula.
7. Repeat steps 1 thru 6 for the second,…

It is possible to calculate all the errors first, then adjust all the weights afterwards.



Backpropagation Training

Off-line (Mini-Batch) Update: Method 3

Iterative minimization of error over training set

- Group training patterns into **batches** = a fraction of the total number of training patterns to be learned.
- For every pattern within a 'batch', calculate the weight gradient (Δw) for each weight in the network. Accumulate the weight gradients(Δw) for each weight. After all the patterns within a batch have made their contributions, all the weights are updated.
- For example, if there were 50,000 training patterns to be learned, an update might happen for every 'batch' of 1000 patterns (1/50 of the total number of training patterns).
- 1 Iteration = equivalent to using all training patterns in a batch once (e.g. 1,000 patterns)
- 1 Epoch = equivalent to using all training patterns in the complete training set once (e.g. 50,000 patterns)

Backpropagation Training

Off-line (Mini-Batch) Update: Method 3

Iterative minimization of error over training set

- Group training patterns into **batches** = a fraction of the total number of training patterns to be learned.
- For every pattern within a 'batch', calculate the weight changes (Δw) for each weight in the network. Accumulate the weight changes (Δw) for each weight. After all the patterns within a batch have made their contributions, all the weights are updated.
- For example, if there were 50,000 training patterns to be learned, an update might happen for every 'batch' of 1000 patterns (1/50 of the total number of training patterns).
- 1 Epoch = equivalent to using all training patterns in the complete training set once
- Neural Network training usually takes many training epochs (e.g. until its performance is at an acceptable level)

Backpropagation Training

Off-line (Mini-Batch) Update: Method 3

Repeat until Epoch > MAX_EPOCHS

For i=1 to NUM_OF_MINI_BATCHES

For each pattern in MiniBatch i

- forward propagate inputs
- compute error deltas at the output units and hidden units
- backpropagate error deltas (needed for computing all weight gradients)
- compute weight gradients and accumulate them, but do not apply them to the weights yet

  e.g.

  $$\Delta W_{jk} = \Delta W_{jk}^{(1)} + \Delta W_{jk}^{(2)} + \Delta W_{jk}^{(3)} + \cdots + \Delta W_{jk}^{(size\ of\ MiniBatch)}$$

End For

Adjust all weights using the computed weight gradients

Compute network performance (P) on Validation Set (or both Training and Test sets)

If (P is at an acceptable level) then End_Training

End For

End Repeat

Backpropagation Training

Off-line (Full-Batch) Update: Method 4

Iterative minimization of error over training set

- For every pattern in the training set, calculate the weight changes (Δw) for each weight in the network. Accumulate the weight changes (Δw) for each weight. After all the patterns have made their contributions, all the weights are updated.

Backpropagation Training

Off-line (Full-Batch) Update: Method 4

Repeat until Epoch > **MAX_EPOCHS**

For each pattern in the **Training Set**

- forward propagate inputs
- compute error deltas at the output units and hidden units
- backpropagate error deltas (needed for computing all weight gradients)
- compute weight gradients and accumulate them, but do not apply them to the weights yet

  e.g.

$$\Delta W_{jk} = \Delta W_{jk}^{(1)} + \Delta W_{jk}^{(2)} + \Delta W_{jk}^{(3)} + \cdots + \Delta W_{jk}^{(number\ of\ patterns)}$$

End For

Adjust all weights using the computed weight gradients

Compute network performance (P) on Validation Set (or both Training and Test sets)

If (P is at an acceptable level) then End_Training

End Repeat

Backpropagation Training

On-line vs. Off-line Update

- The **on-line update/SGD method** is not a “correct” implementation of gradient ascent/descent for the total training error. However, it is often argued that allowing the weights to change immediately after the presentation of an instance of the training set produces a beneficial effect on training – it tends to push the system out of some shallow local minimum.
- The **off-line batch update** method of actually adding up the gradients for all the instances and changing the weights based on that complete gradient is also used and has some advantages for smaller systems. (training set with <= 2,000 patterns).
- For larger nets and larger data sets, the **mini-batch method** has substantial performance advantages. (typical mini-batch size: 64, 128, 256, 512). Benefits from vectorisation using GPUs.